



南開大學  
Nankai University

计算机学院  
计算机系统设计实验报告

PA3 实验报告

姓名：王一诺  
学号：2312331  
专业：计算机科学与技术

2026 年 5 月 24 日

# 目录

|                                            |           |
|--------------------------------------------|-----------|
| <b>1 PA3 实验概述</b>                          | <b>2</b>  |
| <b>2 PA3-1</b>                             | <b>2</b>  |
| 2.1 加载操作系统的第一个用户程序 . . . . .               | 2         |
| 2.2 实现 loader . . . . .                    | 2         |
| 2.3 实现中断机制 . . . . .                       | 3         |
| 2.4 重新组织 TrapFrame 结构体 . . . . .           | 6         |
| 2.5 实现系统调用 . . . . .                       | 8         |
| 2.6 第一阶段总结 . . . . .                       | 11        |
| <b>3 PA3-2</b>                             | <b>12</b> |
| 3.1 在 Nanos-lite 上运行 Hello World . . . . . | 12        |
| 3.2 实现堆区管理 . . . . .                       | 14        |
| 3.3 让 loader 使用文件 . . . . .                | 15        |
| 3.4 实现完整文件系统 . . . . .                     | 18        |
| 3.5 第二阶段总结 . . . . .                       | 21        |
| <b>4 PA3-3</b>                             | <b>22</b> |
| 4.1 把 VGA 显存抽象成文件 . . . . .                | 22        |
| 4.2 把设备输入抽象成文件 . . . . .                   | 25        |
| 4.3 在 NEMU 中运行仙剑奇侠传: . . . . .             | 28        |
| <b>5 思考题与必答题</b>                           | <b>29</b> |
| 5.1 思考题 . . . . .                          | 29        |
| 5.2 必答题 . . . . .                          | 33        |
| 5.2.1 游戏存档读取过程 . . . . .                   | 33        |
| 5.2.2 屏幕更新过程 . . . . .                     | 37        |

## 1 PA3 实验概述

本次 PA3 内容大致包括三个部分：

- 熟悉系统调用，实现中断机制，把“用户程序通过 `int` 指令陷入操作系统，再从操作系统返回用户程序”的完整路径实现出来。
- 在第一阶段基础上，进一步完善系统调用，实现简易文件系统。
- 把设备访问统一到文件系统接口下，将输入输出抽象成文件，运行仙剑奇侠传。

## 2 PA3-1

### 2.1 加载操作系统的第一个用户程序

我们要在 Nanos-lite 上运行的第一个用户程序是 `navy-apps/tests/dummy/dummy.c`。首先我们让 Navy-apps 项目上的程序默认编译到 x86 中：在 `navy-apps/Makefile.check` 中改

```
1  ISA ?= x86
2
```

然后在 `navy-apps/tests/dummy` 下执行 `make`，就会在 `navy-apps/tests/dummy/build/` 目录下生成 `dummy` 的可执行文件。编译 Newlib 时会出现较多 warning，我们可以忽略它们。

在 `nanos-lite/` 目录下执行 `make update`，`nanos-lite/Makefile` 中会将其生成 `ramdisk` 镜像文件 `ramdisk.img`，并包含进 Nanos-lite 成为其中的一部分：

```
wyn@ubuntu:~/Documents/ics2017/nanos-lite$ make update
Building nanos-lite [x86-nemu]
objcopy -S --set-section-flags .bss=alloc,contents -O binary /home/wyn/Documents/ics2017/navy-apps/tests/dummy/build/dummy-x86 build/ramdisk.img
touch src/files.h
ln -sf /home/wyn/Documents/ics2017/navy-apps/libs/libos/src/syscall.h src/syscall.h
```

### 2.2 实现 loader

思路 .

第一阶段的 loader 是 raw program loader。根据实验指导，当前 `ramdisk` 中只有一个用户程序，并且 `nanos-lite/Makefile` 已经通过 `objcopy` 将 ELF 中真正需要执行的代码和数据抽取成了裸二进制文件。因此 loader 暂时不需要解析 ELF 文件头，也不需要遍历 `program header`，只需要把 `ramdisk` 从偏移 0 开始的所有内容复制到约定的用户程序加载地址 `0x4000000`，然后把这个地址作为入口返回即可。

实现过程 .

在 `nanos-lite/src/loader.c` 中保留 `DEFAULT_ENTRY` 为 `0x4000000`。为了读取 `ramdisk`，添加 `ramdisk_read()` 和 `get_ramdisk_size()` 的函数声明。`loader()` 中调用 `get_ramdisk_size()` 获得 `ramdisk` 大小，再调用 `ramdisk_read()` 把全部内容复制到 `DEFAULT_ENTRY`，最后返回 `DEFAULT_ENTRY`。

实现代码如下：

```
1  #define DEFAULT_ENTRY ((void *)0x4000000)
2
3  void ramdisk_read(void *buf, off_t offset, size_t len);
4  size_t get_ramdisk_size();
```



- 第二，在 NEMU 中实现 IDTR 寄存器和 lidt 指令，使 AM 能把 IDT 的基地址和界限写入 CPU 状态。
- 第三，实现 int 指令和 raise\_intr()。int 指令读出中断号后调用 raise\_intr()，raise\_intr() 根据 IDTR 和中断号索引 IDT，取出门描述符中的入口地址，并模拟硬件压栈 EFLAGS、CS、EIP，然后跳转到异常入口。

### 实现过程

在 nanos-lite/src/main.c 中打开 HAS\_ASYE:

```
1 #define HAS_ASYE
2
```

这样 main() 中的以下逻辑会被编译进去:

```
1 #ifdef HAS_ASYE
2     Log("Initializing interrupt/exception handler...");
3     init_irq();
4 #endif
5
```

init\_irq() 会调用 \_\_asye\_init(), \_\_asye\_init() 在 nexus-am/am/arch/x86-nemu/src/asye.c 中设置 IDT，并通过 set\_idt() 执行 lidt。

在 nemu/include/cpu/reg.h 中给 CPU\_state 增加 cs 和 idtr:

```
1 uint32_t cs;
2
3 struct {
4     uint16_t limit;
5     uint32_t base;
6 } idtr;
7
```

在 nemu/src/monitor/monitor.c 的 restart() 中初始化 CS:

```
1 cpu.cs = 8;
2 cpu.eflags.val = 0x00000002;
3
```

在 nemu/src/cpu/exec/system.c 中实现 lidt:

```
1 make_EHelper(lidt) {
2     assert(id_dest->type == OP_TYPE_MEM);
3     cpu.idtr.limit = vaddr_read(id_dest->addr, 2);
4     cpu.idtr.base = vaddr_read(id_dest->addr + 2, 4);
5
6     print_asm_template1(lidt);
7 }
8
```

lidt 的操作数是内存中的 6 字节数据，低 2 字节是 IDT limit，高 4 字节是 IDT base，因此用 `vaddr_read()` 分别读出并写入 `cpu.idtr`。

在 `nemu/src/cpu/exec/system.c` 中实现 `int`：

```

1  make_EHelper(int) {
2      raise_intr(id_dest->val, *eip);
3
4      print_asm("int %s", id_dest->str);
5
6      #ifdef DIFF_TEST
7      diff_test_skip_nemu();
8      #endif
9  }
10

```

`id_dest->val` 是 `int` 指令的立即数，也就是中断号。这里把 `int` 指令下一条指令的地址 `*eip` 作为返回地址传给 `raise_intr()`。

在 `nemu/src/cpu/intr.c` 中实现 `raise_intr()`：

```

1  void raise_intr(uint8_t NO, vaddr_t ret_addr) {
2      uint32_t desc_addr = cpu.idtr.base + NO * sizeof(GateDesc);
3      assert(desc_addr + sizeof(GateDesc) - 1 <= cpu.idtr.base + cpu.idtr.limit);
4
5      uint32_t low = vaddr_read(desc_addr, 4);
6      uint32_t high = vaddr_read(desc_addr + 4, 4);
7      assert(high & 0x8000);
8
9      rtlreg_t val = cpu.eflags.val;
10     rtl_push(&val);
11     val = cpu.cs;
12     rtl_push(&val);
13     val = ret_addr;
14     rtl_push(&val);
15
16     cpu.cs = (low >> 16) & 0xffff;
17     decoding.jump_eip = (low & 0xffff) | (high & 0xffff0000);
18     decoding.is_jump = 1;
19 }
20

```

简单解释一下实现代码：

`desc_addr = cpu.idtr.base + NO * sizeof(GateDesc)` 用中断号索引 IDT。每个门描述符占 8 字节，因此 0x80 号中断会访问 `IDT[0x80]`。

`vaddr_read(desc_addr, 4)` 和 `vaddr_read(desc_addr + 4, 4)` 分别取出门描述符低 32 位和高 32 位。当前 NEMU 简化了门描述符，但仍然保留 `offset` 和 `present` 位。`assert(high & 0x8000)` 用来检查门描述符是否有效。

根据 i386 异常机制，CPU 进入异常处理时会自动压栈 EFLAGS、CS、EIP。代码中按顺序调用 `rtl_push()`，最终栈顶从低地址到高地址依次是 EIP、CS、EFLAGS，正好供后续 `iret` 恢复。

最后从门描述符中拼出目标地址：

```
1 decoding JMP_EIP = (low & 0xffff) | (high & 0xffff0000);
2
```

并设置 `decoding.is_JMP = 1`，表示当前指令结束后不再顺序执行，而是跳转到 IDT 指定的入口，也就是 `nexus-am/am/arch/x86-nemu/src/trap.S` 中的 `vecsys`。

为了让指令可以被执行，还需要在 `nemu/src/cpu/exec/all-instr.h` 中声明 `helper`，并在 `nemu/src/cpu/exec/exec.c` 中接入 `opcode`。lidt 是 0f 01 /3，int imm8 是 cd ib。

实现效果

重新在 Nanos-lite 上运行 dummy 程序：

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 19:42:04, May 19 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x100ef0, end = 0x1054cc, size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
please implement me
nemu: src/cpu/exec/data-mov.c:22: exec_pusha: Assertion `0' failed.
Makefile:46: recipe for target 'run' failed
make[1]: *** [run] Aborted (core dumped)
make[1]: Leaving directory '/home/wyn/Documents/ics2017/nemu'
/home/wyn/Documents/ics2017/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Error 2
```

触发 `exec_pusha` 说明执行流已经是：

- dummy 执行 `int $0x80`
- NEMU 的 int helper 调用 `raise_intr()`
- `raise_intr()` 通过 IDT 找到 0x80 的入口
- 成功跳转到 Nexus-AM 的 `vecsys`
- `vecsys` 进入 `asm_trap`
- `asm_trap` 执行第一条保存现场指令： `pushal`。

## 2.4 重新组织 TrapFrame 结构体

思路

中断跳转到 `vecsys` 后，硬件只保存了 EFLAGS、CS、EIP，通用寄存器还没有保存。`trap.S` 中的 `asm_trap` 会执行 `pushal`，把通用寄存器全部压入栈中，再把当前 `esp` 作为参数传给 `irq_handle()`。因此 `_RegSet` 必须严格匹配栈上的 `trapframe` 布局，否则 C 代码读取到的 `eax`、`ebx`、`irq` 等字段都会错位。

i386 的 `pushal/pusha` 顺序是依次压入 EAX、ECX、EDX、EBX、原始 ESP、EBP、ESI、EDI。由于栈向低地址增长，`pushal` 执行完后，当前 ESP 指向最后压入的 EDI。因此从低地址到高地址看，`trapframe` 中通用寄存器顺序是 EDI、ESI、EBP、ESP、EBX、EDX、ECX、EAX。

## 实现过程

在 nemu/src/cpu/exec/data-mov.c 中实现 pusha:

```

1  make_EHelper(pusha) {
2      rtlreg_t old_esp = cpu.esp;
3
4      rtl_push(&cpu.eax);
5      rtl_push(&cpu.ecx);
6      rtl_push(&cpu.edx);
7      rtl_push(&cpu.ebx);
8      rtl_push(&old_esp);
9      rtl_push(&cpu.ebp);
10     rtl_push(&cpu.esi);
11     rtl_push(&cpu.edi);
12
13     print_asm("pusha");
14 }
15

```

这里必须先保存 old\_esp, 因为 pusha 压入的 ESP 不是执行过程中变化后的 ESP, 而是执行 pusha 之前的 ESP。

在 nexus-am/am/arch/x86-nemu/include/arch.h 中重排 \_RegSet:

```

1  struct _RegSet {
2      uintptr_t edi, esi, ebp, esp, ebx, edx, ecx, eax;
3      int irq;
4      uintptr_t error_code, eip, cs, eflags;
5  };
6

```

trap.S 的关键代码改动如下:

```

1  vecsys:
2      pushl $0
3      pushl $0x80
4      jmp asm_trap
5
6  asm_trap:
7      pushal
8      pushl %esp
9      call irq_handle
10

```

结合硬件压栈和软件压栈, trapframe 从低地址到高地址依次为:

```

1  edi
2  esi
3  ebp
4  esp
5  ebx

```



```

6     edx
7     ecx
8     eax
9     irq
10    error_code
11    eip
12    cs
13    eflags
14

```

简单解释一下实现代码：

`_RegSet` 的第一个字段必须对应当前 `esp` 指向的位置。`pushal` 结束后 `esp` 指向 `edi`，因此结构体第一个字段是 `edi`。随后依次是 `esi`、`ebp`、`esp`、`ebx`、`edx`、`ecx`、`eax`。

`vecsys` 在进入 `asm_trap` 前先执行 `pushl $0` 和 `pushl $0x80`。因为第二次 `push` 后栈顶是 `0x80`，所以在 `pushal` 之后，通用寄存器区域后面的第一个字段是 `irq`，第二个字段是 `error_code`。

硬件在 `raise_intr()` 中提前压入了 `eip`、`cs`、`eflags`，因此 `error_code` 后面依次是 `eip`、`cs`、`eflags`。

如果 `_RegSet` 的顺序不匹配 `trap.S` 的实际压栈顺序，`irq_handle()` 中的 `tf->irq` 就不能正确读到 `0x80`，系统调用事件也就无法被识别。

## 实现效果

实现正确之后，`irq_handle()` 以及后续代码就可以正确地使用 `trap frame` 了。重新在 `Nanos-lite` 上运行 `dummy` 程序：

```

[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 20:17:07, May 19 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x100ef0, end = 0x1054cc, size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/irq.c,5,do_event] system panic: Unhandled event ID = 8
nemu: HIT BAD TRAP at eip = 0x00100032

```

可见，在 `nanos-lite/src/irq.c` 中的 `do_event()` 函数中触发了 `BAD TRAP`：

`[src/irq.c,5,do_event]` kernel system panic: Unhandled event ID = 8

## 2.5 实现系统调用

### 思路

系统调用由用户程序通过 `int $0x80` 触发。中断机制把执行流带到 `vecsys` 后，`trap.S` 保存现场并调用 `irq_handle()`。`irq_handle()` 会把 `irq` 号为 `0x80` 的 `trapframe` 包装成 `_EVENT_SYSCALL` 事件，然后交给 `nanos-lite` 的 `do_event()`。

因此系统调用的实现需要完成四件事：

- 第一，`do_event()` 识别 `_EVENT_SYSCALL` 并调用 `do_syscall()`。
- 第二，`SYSCALL_ARGx()` 宏能从 `_RegSet` 中取出正确的参数寄存器。
- 第三，在 `do_syscall()` 中实现 `SYS_none` 和 `SYS_exit`。

- 第四，实现 popa 和 iret，使系统调用处理完后可以恢复现场并回到用户程序。

### 实现过程

在 nanos-lite/src/irq.c 中识别系统调用事件：

```

1  _RegSet* do_syscall(_RegSet *r);
2
3  static _RegSet* do_event(_Event e, _RegSet* r) {
4      switch (e.event) {
5          case _EVENT_SYSCALL: return do_syscall(r);
6          default: panic("Unhandled event ID = %d", e.event);
7      }
8
9      return r;
10 }
11

```

在 nexus-am/am/arch/x86-nemu/include/arch.h 中实现系统调用参数宏：

```

1  #define SYSCALL_ARG1(r) ((r)->eax)
2  #define SYSCALL_ARG2(r) ((r)->ebx)
3  #define SYSCALL_ARG3(r) ((r)->ecx)
4  #define SYSCALL_ARG4(r) ((r)->edx)
5

```

这是因为 navy-apps/libs/libos/src/nanos.c 中的 \_\_syscall\_\_() 使用如下内联汇编约定：

```

1  asm volatile("int $0x80": "=a"(ret): "a"(type), "b"(a0), "c"(a1), "d"(a2));
2

```

也就是说，系统调用号在 eax，第 1 个参数在 ebx，第 2 个参数在 ecx，第 3 个参数在 edx，返回值也放回 eax。

在 nanos-lite/src/syscall.c 中实现系统调用分发：

```

1  _RegSet* do_syscall(_RegSet *r) {
2      uintptr_t a[4];
3      a[0] = SYSCALL_ARG1(r);
4      a[1] = SYSCALL_ARG2(r);
5      a[2] = SYSCALL_ARG3(r);
6      a[3] = SYSCALL_ARG4(r);
7
8      switch (a[0]) {
9          case SYS_none: SYSCALL_ARG1(r) = 1; break;
10         case SYS_exit: _halt(a[1]); break;
11         default: panic("Unhandled syscall ID = %d", a[0]);
12     }
13
14     return r;
15 }
16

```

`SYS_none` 什么也不做，只把返回值设置为 1。由于返回值约定放在 `eax` 中，所以直接设置 `SYSCALL_ARG1(r) = 1`。

`SYS_exit` 接收一个退出状态参数，这个参数位于 `ebx`，也就是 `a[1]`。实现时调用 `_halt(a[1])` 结束程序。

在 `nemu/src/cpu/exec/data-mov.c` 中实现 `popa`:

```

1  make_EHelper(popa) {
2      rtl_pop(&t0);
3      rtl_sr_l(R_EDI, &t0);
4      rtl_pop(&t0);
5      rtl_sr_l(R_ESI, &t0);
6      rtl_pop(&t0);
7      rtl_sr_l(R_EBP, &t0);
8      rtl_pop(&t0);
9      rtl_pop(&t0);
10     rtl_sr_l(R_EBX, &t0);
11     rtl_pop(&t0);
12     rtl_sr_l(R_EDX, &t0);
13     rtl_pop(&t0);
14     rtl_sr_l(R_ECX, &t0);
15     rtl_pop(&t0);
16     rtl_sr_l(R_EAX, &t0);
17
18     print_asm("popa");
19 }
20

```

`popa` 的恢复顺序与 `pusha` 相反，但需要注意：栈中保存的 `ESP` 不能被弹回 `ESP`，所以中间对 `ESP` 对应的槽位只执行 `rtl_pop(&t0)`，不写回 `R_ESP`。

在 `nemu/src/cpu/exec/system.c` 中实现 `iret`:

```

1  make_EHelper(iret) {
2      rtl_pop(&t0);
3      decoding.jump_eip = t0;
4      rtl_pop(&t0);
5      cpu.cs = t0;
6      rtl_pop(&t0);
7      cpu.eflags.val = t0;
8      decoding.is_jump = 1;
9
10     print_asm("iret");
11 }
12

```

`iret` 从栈顶依次弹出 `EIP`、`CS`、`EFLAGS`，并跳回 `EIP` 指向的位置。因此系统调用处理完成后，用户程序会从 `int $0x80` 的下一条指令继续执行，并从 `eax` 中获得返回值。

## 实现效果

实现成功后, 再次运行 dummy 程序, 会看到 GOOD TRAP 的信息:

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 20:17:07, May 19 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x100fd8, end = 0x1055b4, size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
nemu: HIT GOOD TRAP at eip = 0x00100032
```

## 2.6 第一阶段总结

第一阶段完成后, 从 Nanos-lite 加载 dummy 到 dummy 正常退出, 整体流程如下:

```
1  Nanos-lite main()
2  -> init_ramdisk()
3  -> init_device()
4  -> init_irq()
5      -> _asye_init()
6      -> set_idt()
7      -> lidt
8  -> loader()
9      -> ramdisk_read(0x4000000, 0, ramdisk_size)
10 -> jump to 0x4000000
11 -> dummy main()
12 -> _syscall_(SYS_none, 0, 0, 0)
13     -> int $0x80
14     -> raise_intr(0x80, next_eip)
15     -> vecsys
16     -> asm_trap
17     -> pushal
18     -> irq_handle()
19     -> do_event(_EVENT_SYSCALL)
20     -> do_syscall()
21     -> return value written to eax
22     -> popal
23     -> iret
24 -> dummy receives return value 1
25 -> dummy returns 0
26 -> exit(0)
27     -> SYS_exit
28     -> _halt(0)
29     -> GOOD TRAP
30
```

这一阶段的核心是把“用户程序通过 int 指令陷入操作系统, 再从操作系统返回用户程序”的完整路径实现出来。loader 负责把用户程序放到正确位置; 中断机制负责从 int 指令跳到内核入口; trapframe 负责保存和恢复现场; 系统调用处理负责根据 eax 中的系统调用号完成具体服务。

## 3 PA3-2

### 3.1 在 Nanos-lite 上运行 Hello World

思路 .

PA3 第一阶段已经打通了系统调用的基本路径：用户程序通过 `int $0x80` 陷入 Nanos-lite，Nanos-lite 根据系统调用号进行分发，然后通过 `iret` 返回用户程序。第二阶段首先要让 `hello` 程序能够输出内容，因此需要实现 `write()` 系统调用。

Navy-apps 中的用户程序调用 `write()` 时，最终会进入 `navy-apps/libs/libos/src/nanos.c` 中的 `_write()`。`_write()` 需要通过统一的 `_syscall_()` 接口触发 `SYS_write`。Nanos-lite 收到 `SYS_write` 后，根据参数 `fd`、`buf`、`count` 完成输出。当前阶段只要求标准输出和标准错误，`fd` 为 1 或 2 时，Nanos-lite 直接调用 AM 提供的 `_putc()` 将字符输出到串口。

实现过程 .

首先在用户态 `libos` 中实现 `_write()`，使它不再直接调用 `_exit(SYS_write)`，而是真正发起系统调用：

```
1 int _write(int fd, void *buf, size_t count){
2     return _syscall_(SYS_write, fd, (uintptr_t)buf, count);
3 }
4
```

`_syscall_()` 的实现中，系统调用号放入 `eax`，第一个参数放入 `ebx`，第二个参数放入 `ecx`，第三个参数放入 `edx`，然后执行 `int $0x80`：

```
1 int _syscall_(int type, uintptr_t a0, uintptr_t a1, uintptr_t a2){
2     int ret = -1;
3     asm volatile("int $0x80": "=a"(ret): "a"(type), "b"(a0), "c"(a1), "d"(a2));
4     return ret;
5 }
6
```

然后在 Nanos-lite 的 `do_syscall()` 中添加 `SYS_write` 分支：

```
1 case SYS_write:
2     SYSCALL_ARG1(r) = fs_write(a[1], (const void *)a[2], a[3]);
3     break;
4
```

这里 `a[1]` 是 `fd`，`a[2]` 是用户传入的缓冲区地址，`a[3]` 是写入长度。`fs_write()` 返回实际写入的字节数，再写回 `eax`，作为 `write()` 的返回值。

在文件系统中实现 `stdout/stderr` 的写入：

```
1 ssize_t fs_write(int fd, const void *buf, size_t len) {
2     assert(fd >= 0 && (size_t)fd < NR_FILES);
3
4     if (fd == FD_STDOUT || fd == FD_STDERR) {
5         const char *p = buf;
6         for (size_t i = 0; i < len; i++) {
```

```
7     _putc(p[i]);
8   }
9   return len;
10  }
11
12  ...
13  }
14
```

最后，为了让 Nanos-lite 加载 hello 程序，在阶段 2 要求 1 中临时修改 nanos-lite/Makefile，把 raw ramdisk 中的唯一程序从 dummy 换成 hello：

```
1 OBJCOPY_FILE = $(NAVY_HOME)/tests/hello/build/hello-x86
2
```

此时 update 规则仍然可以使用 raw ramdisk 方式：

```
1 update: update-ramdisk-objcopy src/syscall.h
2     @touch src/initrd.S
3
```

简单解释一下实现代码：

用户程序 `write(1, "Hello World!\n", 13)` 会调用 `_write()`，`_write()` 通过 `SYS_write` 进入内核。Nanos-lite 的 `do_syscall()` 从 `trapframe` 中取出 `eax/ebx/ecx/edx`，识别出系统调用号为 `SYS_write` 后调用 `fs_write()`。

`fs_write()` 中对 `fd` 做区分。`fd` 为 `stdout` 或 `stderr` 时，它们并不对应 ramdisk 中的普通文件，而是特殊输出设备，因此直接调用 `_putc()`。`write()` 的语义是返回成功写入的字节数，因此 `fs_write()` 输出 `len` 个字符后返回 `len`。

实现后，hello 程序可以先通过 `write()` 输出 "Hello World!"，之后 `printf()` 也可以通过 `write()` 输出格式化后的字符串。

### 实现效果

重新编译 Nanos-lite 并运行：

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 20:17:07, May 19 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x1015c0, end = 0x105ce0, size = 18208 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
Hello World!
Hello World for the 2th time
Hello World for the 3th time
Hello World for the 4th time
Hello World for the 5th time
Hello World for the 6th time
Hello World for the 7th time
Hello World for the 8th time
Hello World for the 9th time
Hello World for the 10th time
Hello World for the 11th time
Hello World for the 12th time
Hello World for the 13th time
Hello World for the 14th time
Hello World for the 15th time
Hello World for the 16th time
Hello World for the 17th time
```

### 3.2 实现堆区管理

思路 .

hello 程序中除了直接调用 `write()`，还会调用 `printf()`。`printf()` 第一次使用时，Newlib 会尝试通过 `malloc()` 申请缓冲区，用来保存格式化后的字符串。`malloc()` 管理用户程序的堆区，而堆区边界需要通过 `sbrk()/brk()` 调整。

在 Navy-apps 的 Newlib 中，`sbrk()` 最终会调用 `libos` 中的 `__sbrk()`。因此需要在用户态实现 `__sbrk()`，并在 Nanos-lite 中实现 `SYS_brk` 系统调用。当前 Nanos-lite 是单任务操作系统，还没有真正的内存隔离和进程地址空间管理，所以 `SYS_brk` 可以直接返回 0，表示调整 program break 成功。

实现过程 .

在 Nanos-lite 的 `do_syscall()` 中添加 `SYS_brk`:

```
1 case SYS_brk:
2     SYSCALL_ARG1(r) = 0;
3     break;
4
```

在用户态 `libos` 中实现 `__sbrk()`。program break 初始值使用链接器提供的 `__end` 符号，表示用户程序数据段结束的位置。每次调用 `__sbrk(increment)` 时，计算新的 program break，并通过 `SYS_brk` 请求内核设置。如果内核返回 0，说明成功，更新本地记录并返回旧的 program break。

```
1 void *__sbrk(intptr_t increment){
2     extern char __end;
3     static uintptr_t program_break = (uintptr_t)&__end;
4
5     uintptr_t old_break = program_break;
6     uintptr_t new_break = program_break + increment;
7     int ret = __syscall__(SYS_brk, new_break, 0, 0);
8     if (ret == 0) {
```

```

9     program_break = new_break;
10    return (void *)old_break;
11  }
12  return (void *)-1;
13  }
14

```

`_end` 是用户程序链接时产生的符号，表示静态数据段结束位置。程序刚启动时，堆区大小为 0，因此 `program_break` 初始化为 `&_end`。

`_sbrk()` 的返回值不是新的 program break，而是旧的 program break。`malloc()` 会把旧 program break 到新 program break 之间的空间纳入堆区管理。

`SYS_brk` 当前只返回 0，因为 Nanos-lite 暂时没有实现真正的内存分配边界检查。这样 Newlib 认为堆区扩展成功，`printf()` 就能申请缓冲区，把格式化后的字符串集中起来，再通过 `write()` 输出。

实现前后屏幕上的输出内容完全一样，此处不再重复展示。区别在内部行为：未实现 `_sbrk()` 时，`printf()` 可能退化成多次小 `write`，甚至逐字符输出；实现 `_sbrk()` 后，`printf()` 可以使用缓冲区，会以更少次数的 `write` 输出完整字符串。

### 3.3 让 loader 使用文件

思路 .

第一阶段的 loader 是 raw program loader，直接把 ramdisk 的全部内容复制到 `0x4000000`。这种方式只适合 ramdisk 中只有一个用户程序的情况。第二阶段开始引入文件系统，ramdisk 中会包含多个文件，例如 `/bin/hello`、`/bin/text`、`/share/texts/num`。此时 loader 不能再假设 ramdisk 的全部内容就是一个程序，而应该通过文件系统按文件名打开目标程序并加载。

因此需要实现 `fs_open()`、`fs_read()`、`fs_close()`，并增加 `fs_filesz()` 用来获得文件大小。loader 根据文件名打开文件，把该文件内容读取到 `0x4000000`，然后关闭文件并返回入口地址。

实现过程 .

文件表中每一项增加 `open_offset`，用来记录当前文件读写偏移：

```

1  typedef struct {
2      char *name;
3      size_t size;
4      off_t disk_offset;
5      off_t open_offset;
6  } Finfo;
7

```

实现 `fs_open()`：

```

1  int fs_open(const char *pathname, int flags, int mode) {
2      for (size_t fd = 0; fd < NR_FILES; fd++) {
3          if (strcmp(pathname, file_table[fd].name) == 0) {
4              file_table[fd].open_offset = 0;
5              return fd;
6          }
7      }

```



```

8     panic("file not found: %s", pathname);
9     return -1;
10 }
11

```

实现 fs\_filesz():

```

1 size_t fs_filesz(int fd) {
2     assert(fd >= 0 && (size_t)fd < NR_FILES);
3     return file_table[fd].size;
4 }
5

```

实现 fs\_read() 的普通文件读取:

```

1 ssize_t fs_read(int fd, void *buf, size_t len) {
2     assert(fd >= 0 && (size_t)fd < NR_FILES);
3
4     if (fd < FD_NORMAL) {
5         return 0;
6     }
7
8     Finfo *file = &file_table[fd];
9     assert(file->open_offset >= 0);
10    size_t open_offset = (size_t)file->open_offset;
11    assert(open_offset <= file->size);
12    if (len > file->size - open_offset) {
13        len = file->size - open_offset;
14    }
15
16    ramdisk_read(buf, file->disk_offset + file->open_offset, len);
17    file->open_offset += len;
18    return len;
19 }
20

```

实现 fs\_close():

```

1 int fs_close(int fd) {
2     assert(fd >= 0 && (size_t)fd < NR_FILES);
3     return 0;
4 }
5

```

修改 loader:

```

1 #define DEFAULT_ENTRY ((void *)0x4000000)
2
3 uintptr_t loader(_Protect *as, const char *filename) {
4     int fd = fs_open(filename, 0, 0);
5

```

```

5     fs_read(fd, DEFAULT_ENTRY, fs_filesz(fd));
6     fs_close(fd);
7     return (uintptr_t)DEFAULT_ENTRY;
8 }
9

```

修改 main(), 通过文件名指定要加载的用户程序:

```

1  #define DEFAULT_PROGRAM "/bin/text"
2
3  uint32_t entry = loader(NULL, DEFAULT_PROGRAM);
4

```

为了让 ramdisk 中包含多个文件, 需要把 nanos-lite/Makefile 中 update 规则切换为完整 fsimg:

```

1  update: update-ramdisk-fsimg src/syscall.h
2      @touch src/initrd.S
3

```

简单解释一下实现代码:

update-ramdisk-fsimg 会把 Navy-apps 的 fsimg 目录中的文件打包到 ramdisk, 并生成 src/files.h。files.h 中保存每个文件的文件名、大小和在 ramdisk 中的偏移。例如 /bin/hello 和 /bin/text 都会成为 file\_table 中的普通文件项。

loader 调用 fs\_open(filename, 0, 0) 后得到文件描述符 fd。fs\_filesz(fd) 返回该程序文件大小。fs\_read() 根据 file\_table[fd].disk\_offset 找到程序在 ramdisk 中的位置, 并读取到 0x4000000。

### 实现效果

这样以后切换用户程序不需要修改 loader, 也不需要修改 ramdisk 的 raw 文件来源, 只需要修改 DEFAULT\_PROGRAM 的文件名:

#define DEFAULT\_PROGRAM "/bin/hello" , update 后再 make run 是:

```

[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,21,main] 'Hello World!' from Nanos-lite
[src/main.c,22,main] Build time: 02:04:39, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101bd0, end = 0x37061a, size = 2550346 bytes
[src/main.c,29,main] Initializing interrupt/exception handler...
Hello World!
Hello World for the 2th time
Hello World for the 3th time
Hello World for the 4th time
Hello World for the 5th time
Hello World for the 6th time
Hello World for the 7th time
Hello World for the 8th time
Hello World for the 9th time
Hello World for the 10th time

```

#define DEFAULT\_PROGRAM "/bin/text" ,(同时完成后面的“4. 实现完整文件系统”后)update 后再 make run 是:

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,21,main] 'Hello World!' from Nanos-lite
[src/main.c,22,main] Build time: 02:06:59, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101bd0, end = 0x37061a, size = 2550346 bytes
[src/main.c,29,main] Initializing interrupt/exception handler...
PASS!!!
nemu: HIT GOOD TRAP at eip = 0x00100032
```

只要对应程序已经被编译并安装到 fsimg/bin 中，loader 就能通过文件系统加载它。

### 3.4 实现完整文件系统

思路 .

为了运行 /bin/text 测试程序，需要支持普通文件读写和文件定位。text 程序会打开 /share/texts/num，使用 fseek()/ftell()/fscanf()/fprintf() 测试文件大小、读取、写入和偏移调整。因此 Nanos-lite 需要实现 fs\_write() 和 fs\_lseek()，并在系统调用层实现 SYS\_open、SYS\_read、SYS\_write、SYS\_close、SYS\_lseek。

这个简易文件系统中，文件数量和大小固定，所有普通文件连续存放在 ramdisk 中。每个文件通过 file\_table 中的 size 和 disk\_offset 定位。open\_offset 表示当前读写位置，每次 read/write 后自动前进，lseek() 则显式修改 open\_offset。

实现过程 .

在 Nanos-lite 中添加文件系统相关系统调用分发：

```
1  case SYS_open:
2      SYSCALL_ARG1(r) = fs_open((const char *)a[1], a[2], a[3]);
3      break;
4  case SYS_read:
5      SYSCALL_ARG1(r) = fs_read(a[1], (void *)a[2], a[3]);
6      break;
7  case SYS_write:
8      SYSCALL_ARG1(r) = fs_write(a[1], (const void *)a[2], a[3]);
9      break;
10 case SYS_close:
11     SYSCALL_ARG1(r) = fs_close(a[1]);
12     break;
13 case SYS_lseek:
14     SYSCALL_ARG1(r) = fs_lseek(a[1], a[2], a[3]);
15     break;
16
```

在用户态 libos 中实现系统调用封装：

```
1  int _open(const char *path, int flags, mode_t mode) {
2      return _syscall(SYS_open, (uintptr_t)path, flags, mode);
3  }
4
```

```

5 int _read(int fd, void *buf, size_t count) {
6     return _syscall_(SYS_read, fd, (uintptr_t)buf, count);
7 }
8
9 int _close(int fd) {
10    return _syscall_(SYS_close, fd, 0, 0);
11 }
12
13 off_t _lseek(int fd, off_t offset, int whence) {
14    return _syscall_(SYS_lseek, fd, offset, whence);
15 }
16

```

实现普通文件写入:

```

1 ssize_t fs_write(int fd, const void *buf, size_t len) {
2     assert(fd >= 0 && (size_t)fd < NR_FILES);
3
4     if (fd == FD_STDOUT || fd == FD_STDERR) {
5         const char *p = buf;
6         for (size_t i = 0; i < len; i++) {
7             _putc(p[i]);
8         }
9         return len;
10    }
11
12    if (fd < FD_NORMAL) {
13        return 0;
14    }
15
16    Finfo *file = &file_table[fd];
17    assert(file->open_offset >= 0);
18    size_t open_offset = (size_t)file->open_offset;
19    assert(open_offset <= file->size);
20    if (len > file->size - open_offset) {
21        len = file->size - open_offset;
22    }
23
24    ramdisk_write(buf, file->disk_offset + file->open_offset, len);
25    file->open_offset += len;
26    return len;
27 }
28

```

实现文件定位:

```

1 off_t fs_lseek(int fd, off_t offset, int whence) {
2     assert(fd >= 0 && (size_t)fd < NR_FILES);
3

```

```

4     Finfo *file = &file_table[fd];
5     off_t new_offset = 0;
6     switch (whence) {
7         case SEEK_SET: new_offset = offset; break;
8         case SEEK_CUR: new_offset = file->open_offset + offset; break;
9         case SEEK_END: new_offset = (off_t)file->size + offset; break;
10        default: panic("invalid whence = %d", whence);
11    }
12
13    assert(new_offset >= 0 && new_offset <= (off_t)file->size);
14    file->open_offset = new_offset;
15    return new_offset;
16 }
17

```

简单解释一下实现代码：

fs\_write() 分为特殊文件和普通文件两类。stdout/stderr 是特殊输出文件，直接输出到串口。普通文件则通过 file\_table 找到 ramdisk 中的真实位置。写入地址为：

```

1     file->disk_offset + file->open_offset
2

```

写入后 open\_offset 增加实际写入长度。由于文件大小固定，如果写入长度越过文件末尾，就把 len 截断到剩余空间，防止越界。

fs\_lseek() 根据 whence 计算新偏移。SEEK\_SET 表示从文件开头计算，SEEK\_CUR 表示从当前 open\_offset 计算，SEEK\_END 表示从文件末尾计算。计算出的 new\_offset 必须在 [0, file->size] 范围内。

text 程序的测试流程大致如下：

```

1     open("/share/texts/num", "r+")
2     seek to end
3     check file size == 5000
4     seek to middle
5     read and check numbers
6     seek to beginning
7     write new numbers
8     seek/read again
9     check modified content
10    print "PASS!!!"
11

```

如果 open/read/write/lseek/close 都正确实现，/bin/text 会输出 PASS!!!，说明阶段 2 的普通文件系统功能通过测试。

实现效果

运行测试程序/bin/text:

```

[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,21,main] 'Hello World!' from Nanos-lite
[src/main.c,22,main] Build time: 02:06:59, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101bd0, end = 0x37061a, size = 2550346 bytes
[src/main.c,29,main] Initializing interrupt/exception handler...
PASS!!!
nemu: HIT GOOD TRAP at eip = 0x00100032

```

输出 PASS!!!，通过测试。

### 3.5 第二阶段总结

第二阶段完成后，系统能力从“只能加载一个裸程序并处理最简单的系统调用”扩展为“可以从 ramdisk 文件系统中加载指定程序，并支持标准输出、堆区扩展和普通文件读写”。

整体流程如下：

```

1  make update
2      -> build Navy-apps fsimg
3      -> objcopy executable files to raw binary
4      -> concatenate fsimg files into ramdisk.img
5      -> generate nanos-lite/src/files.h
6
7  Nanos-lite main()
8      -> init_ramdisk()
9      -> init_irq()
10     -> init_fs()
11     -> loader(NULL, DEFAULT_PROGRAM)
12     -> fs_open(DEFAULT_PROGRAM)
13     -> fs_read(program file, 0x4000000, file size)
14     -> fs_close(program file)
15     -> jump to user program
16
17  User program
18     -> open/read/write/lseek/brk
19     -> _syscall_()
20     -> int $0x80
21     -> do_event(_EVENT_SYSCALL)
22     -> do_syscall()
23     -> fs_* or SYS_brk handling
24     -> return to user program
25

```

这一阶段的核心是把 ramdisk 从“一个裸程序镜像”提升为“多个文件组成的简易文件系统”，并让用户态 C 库通过系统调用使用这些服务。

## 4 PA3-3

本阶段的目标是把设备也纳入文件系统抽象中，使用户程序不再直接接触底层 IOE 接口，而是通过标准文件接口访问显示设备、屏幕信息和输入事件。第三阶段主要实现了两个要求：把 VGA 显存抽象成文件，以及把设备输入抽象成文件。

### 4.1 把 VGA 显存抽象成文件

思路

Navy-apps 和 Nanos-lite 约定使用 `/dev/fb` 表示帧缓冲区。用户程序把 `/dev/fb` 看成一个按行优先排列的一维字节数组，每个像素占 4 字节，颜色格式为 00rrggbb。程序只需要通过 `lseek()` 移动文件偏移，再通过 `write()` 写入像素数据，就可以更新屏幕上的指定区域。

同时，用户程序需要知道屏幕宽度和高度，因此 Nanos-lite 还需要提供 `/proc/dispinfo`。它是一个只读的 `procfs` 风格文件，内容是文本形式的键值对，例如 `WIDTH` 和 `HEIGHT`。用户程序通过 `open()`、`read()` 读取这个文件，再解析出屏幕尺寸。

这两个文件虽然出现在文件记录表 `file_table` 中，但它们并不真实存在于 `ramdisk` 中，所以不能走普通文件的 `ramdisk_read()` 和 `ramdisk_write()` 逻辑，而应该在 `fs_read()` 和 `fs_write()` 中根据文件描述符进行重定向。

整体设计如下：

```

1  初始化设备：
2      调用 _ioe_init()
3      从 _screen.width 和 _screen.height 取得屏幕大小
4      生成 /proc/dispinfo 对应的字符串
5
6  初始化文件系统：
7      将 /dev/fb 的文件大小设置为 width * height * 4
8
9  读取 /proc/dispinfo：
10     根据 open_offset 从 dispinfo 字符串中复制数据
11     返回实际复制的字节数
12
13 写入 /dev/fb：
14     将字节偏移 offset 转换为像素偏移
15     将像素偏移转换为屏幕坐标 x, y
16     按行调用 _draw_rect() 绘制像素
17

```

实现过程

首先，在 `nanos-lite/src/device.c` 中实现 `init_device()`。`_ioe_init()` 初始化 IOE 后，AM 会提供全局变量 `_screen`，其中保存屏幕宽度和高度。利用 `snprintf()` 把这些信息提前写入静态字符数组 `dispinfo` 中。

关键代码如下：

```

1  void init_device() {
2      _ioe_init();

```

```

3     snprintf(dispinfo, sizeof(dispinfo), "WIDTH: %d\nHEIGHT: %d\n",
4         _screen.width, _screen.height);
5 }
6

```

这段代码的作用是生成 /proc/dispinfo 的文件内容。Navy-apps 中的 NDL 会打开 /proc/dispinfo，并用冒号分割键和值，因此这里输出 WIDTH 和 HEIGHT 两行文本即可。

然后，在 nanos-lite/src/fs.c 的 init\_fs() 中初始化 /dev/fb 的大小。因为每个像素占 4 字节，所以 /dev/fb 的大小等于屏幕宽度乘以屏幕高度再乘以 sizeof(uint32\_t)。

关键代码如下：

```

1 void init_fs() {
2     file_table[FD_FB].size = _screen.width * _screen.height * sizeof(uint32_t);
3 }
4

```

这样做之后，/dev/fb 就具有了和普通文件类似的 size 信息，fs\_lseek() 可以根据这个大小检查偏移范围。用户程序通过 lseek() 设置偏移时，就不会移动到帧缓冲范围之外。

接着，实现 dispinfo\_read()。它从 dispinfo 字符串的 offset 位置开始，把最多 len 字节复制到用户提供的 buf 中。如果 offset 已经超过字符串长度，则返回 0；如果 len 超过剩余长度，则截断为剩余长度。

关键代码如下：

```

1 size_t dispinfo_read(void *buf, off_t offset, size_t len) {
2     size_t size = strlen(dispinfo);
3     if (offset < 0 || (size_t)offset >= size) {
4         return 0;
5     }
6
7     if (len > size - (size_t)offset) {
8         len = size - (size_t)offset;
9     }
10    memcpy(buf, dispinfo + offset, len);
11    return len;
12 }
13

```

这里返回实际读取的字节数，便于 fs\_read() 正确更新文件当前偏移 open\_offset。这样用户程序可以像读取普通文件一样多次读取 /proc/dispinfo。

然后，实现 fb\_write()。/dev/fb 是字节序列，但 \_draw\_rect() 接收的是像素数组和二维坐标，因此需要先完成偏移转换。

转换关系如下：

```

1 pixel_offset = offset / 4
2 x = pixel_offset % screen_width
3 y = pixel_offset / screen_width
4

```



由于一次 write() 可能从屏幕中间开始，写入长度也可能跨越多行，而 \_draw\_rect() 要求传入的是一个矩形区域的像素。为了保证行边界正确，我采用按行拆分的方式，每次只绘制高度为 1 的一行。关键代码如下：

```

1 void fb_write(const void *buf, off_t offset, size_t len) {
2     assert(offset >= 0);
3     assert(offset % sizeof(uint32_t) == 0);
4     assert(len % sizeof(uint32_t) == 0);
5
6     const uint32_t *pixels = buf;
7     size_t pixel_offset = (size_t)offset / sizeof(uint32_t);
8     size_t nr_pixels = len / sizeof(uint32_t);
9
10    while (nr_pixels > 0) {
11        int x = pixel_offset % _screen.width;
12        int y = pixel_offset / _screen.width;
13        if (y >= _screen.height) {
14            break;
15        }
16
17        size_t row_pixels = _screen.width - x;
18        if (row_pixels > nr_pixels) {
19            row_pixels = nr_pixels;
20        }
21        _draw_rect(pixels, x, y, row_pixels, 1);
22
23        pixels += row_pixels;
24        pixel_offset += row_pixels;
25        nr_pixels -= row_pixels;
26    }
27    _draw_sync();
28 }
29

```

这段代码先检查 offset 和 len 是否都是 4 字节对齐，因为一个像素占 4 字节。随后把 offset 转为像素偏移，再转成 x 和 y。row\_pixels 表示当前行还能写多少个像素，如果剩余像素数不足一整行，就只写剩余像素。每次调用 \_draw\_rect() 后，更新像素指针和像素偏移，直到所有像素写完。

最后，在 fs\_read() 和 fs\_write() 中加入特殊文件的分流逻辑。读取 /proc/dispinfo 时调用 dispinfo\_read()，写入 /dev/fb 时调用 fb\_write()，普通文件仍然走 ramdisk。

读取 /proc/dispinfo 的逻辑如下：

```

1 if (fd == FD_DISPINFO) {
2     Finfo *file = &file_table[fd];
3     size_t ret = dispinfo_read(buf, file->open_offset, len);
4     file->open_offset += ret;
5     return ret;
6 }
7

```

写入 /dev/fb 的逻辑如下:

```

1  if (fd == FD_FB) {
2      Finfo *file = &file_table[fd];
3      assert(file->open_offset >= 0);
4      size_t open_offset = (size_t)file->open_offset;
5      assert(open_offset <= file->size);
6      if (len > file->size - open_offset) {
7          len = file->size - open_offset;
8      }
9      fb_write(buf, file->open_offset, len);
10     file->open_offset += len;
11     return len;
12 }
13

```

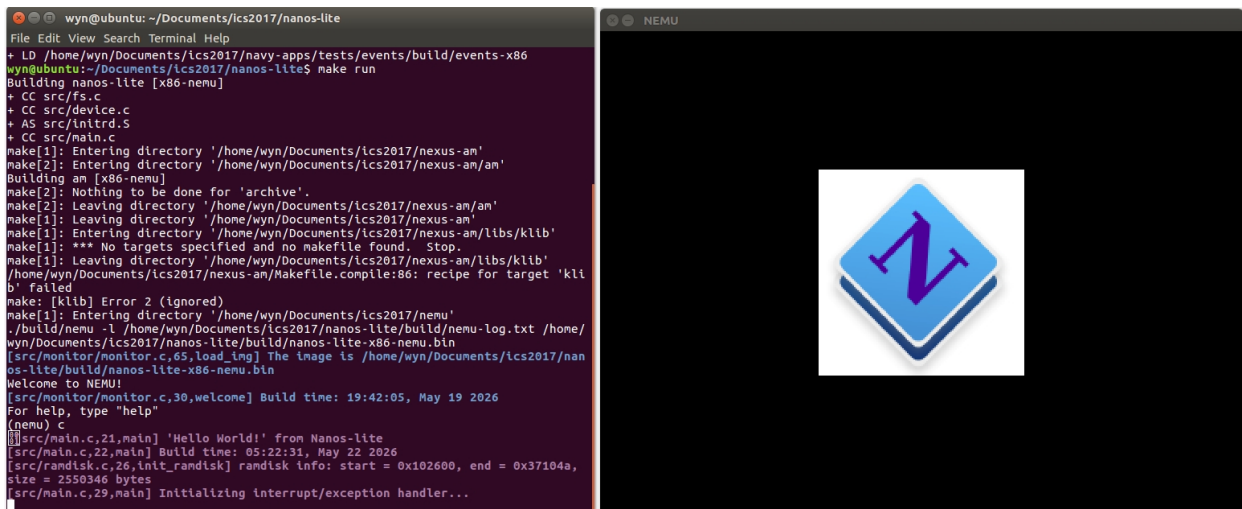
这里先根据 /dev/fb 的文件大小裁剪写入长度, 避免超过屏幕帧缓冲范围。然后调用 fb\_write() 完成实际绘制, 并更新 open\_offset。

实现效果

验证 /dev/fb 时, 把 main.c (line 14) 的 DEFAULT\_PROGRAM 临时改成:

```
#define DEFAULT_PROGRAM "/bin/bmptest"
```

让 Nanos-lite 加载 /bin/bmptest:



可见, 屏幕上显示了 ProjectN 的 Logo, 测试通过。

## 4.2 把设备输入抽象成文件

思路

输入设备包括键盘和时钟。Nanos-lite 和 Navy-apps 约定用 /dev/events 抽象这些输入事件。/dev/events 是一个只读设备文件, 每次读取返回一个以换行符结束的文本事件。

事件格式如下:

```
1  t 1234
```

```
kd RETURN
ku A
```

其中 t 表示时钟事件，后面的数字是系统启动后的毫秒数；kd 表示按键按下，ku 表示按键松开，后面跟按键名。

AM 提供 `_read_key()` 用于读取键盘事件。如果没有键盘事件，返回 `_KEY_NONE`；如果有键盘事件，则返回按键码。按键按下事件会带有 0x8000 标志位，按键松开事件不带该标志位。因此需要用 0x8000 判断事件类型，再清除这个标志位得到真正的按键编号。

由于时钟事件随时都能生成，如果总是先返回时钟事件，键盘事件可能一直得不到处理。因此 `events_read()` 必须优先处理键盘事件；只有在没有键盘事件时，才返回时钟事件。

整体设计如下：

```
读取 /dev/events:
key = _read_key()
if key != _KEY_NONE:
    如果 key 带有 0x8000, 则事件类型为 kd
    否则事件类型为 ku
    清除 0x8000 后得到按键编号
    通过 keyname 数组得到按键名
    生成 "kd KEY\n" 或 "ku KEY\n"
else:
    调用 _uptime()
    生成 "t TIME\n"
将事件字符串复制到 buf 中
返回实际复制的长度
```

## 实现过程

首先，在 `nanos-lite/src/device.c` 中实现 `events_read()`。函数内部先调用 `_read_key()`，判断当前是否有键盘事件。

关键代码如下：

```
size_t events_read(void *buf, size_t len) {
    char event[32];
    int key = _read_key();

    if (key != _KEY_NONE) {
        bool down = (key & 0x8000) != 0;
        key &= ~0x8000;
        snprintf(event, sizeof(event), "%s %s\n", down ? "kd" : "ku", keyname[key]);
    } else {
        snprintf(event, sizeof(event), "t %d\n", (int)_uptime());
    }

    size_t n = strlen(event);
    if (n > len) {
```

```

15     n = len;
16 }
17 memcpy(buf, event, n);
18 return n;
19 }
20

```

这段代码中，down 用于记录是否为按下事件。key &= 0x8000 的作用是去掉事件类型标志，只保留按键编号。keyname 数组已经由 \_KEYS 宏生成，能够把按键编号转换成字符串，例如 RETURN、A、SPACE 等。

如果没有按键事件，就调用 \_uptime() 获取系统启动后的时间，并生成时钟事件。最后根据 len 限制复制长度，保证最多只向 buf 写入 len 字节。

然后，在 nanos-lite/src/fs.c 的 fs\_read() 中加入 /dev/events 的分流逻辑。

关键代码如下：

```

1  if (fd == FD_EVENTS) {
2      return events_read(buf, len);
3  }
4

```

/dev/events 是设备输入流，不对应 ramdisk 中的实体文件，也不依赖 open\_offset。每次 read() 都直接向设备查询当前事件，并返回事件字符串。

最后，为了验证输入设备文件，需要让 Nanos-lite 加载 /bin/events。该程序会循环读取 /dev/events，并把读取到的事件打印出来。

main.c 中最终默认加载程序设置为：

```

1  #define DEFAULT_PROGRAM "/bin/events"
2

```

## 实现效果

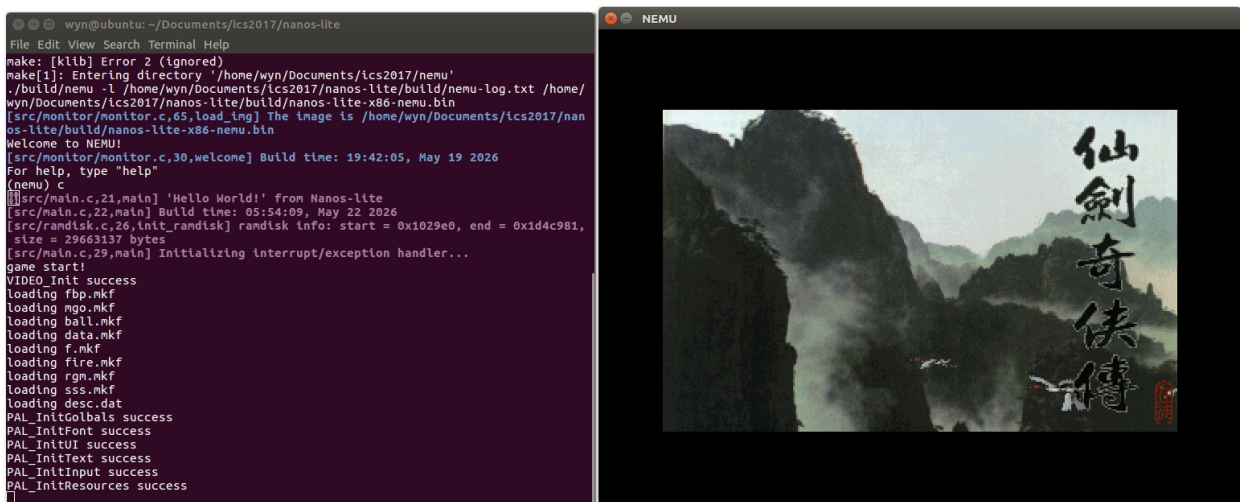
让 Nanos-lite 加载 /bin/events:

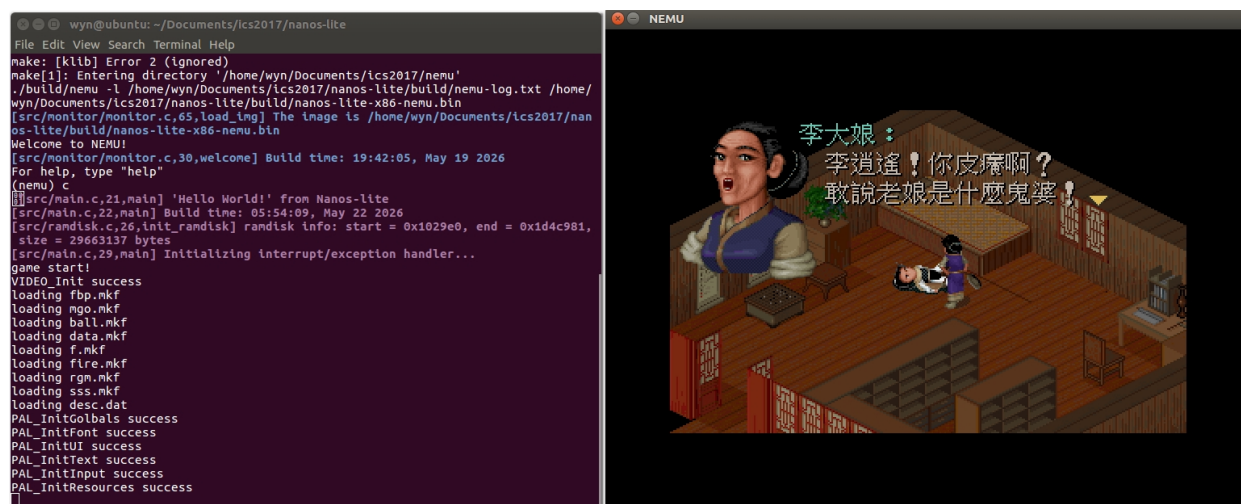
```
wyn@ubuntu: ~/Documents/ics2017/nanos-lite
File Edit View Search Terminal Help
2017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:42:05, May 19 2026
For help, type "help"
(nemu) c
[src/main.c,21,main] 'Hello World!' from Nanos-lite
[src/main.c,22,main] Build time: 05:26:25, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102600, end = 0x37104a, size = 2550346 bytes
[src/main.c,29,main] Initializing interrupt/exception handler...
receive event: t 678
receive event: t 1215
receive event: t 1781
receive event: t 2314
receive event: t 2885
receive event: t 3430
receive event: kd W
receive event: ku W
receive event: kd A
receive event: ku A
receive event: t 6236
receive event: kd C
receive event: ku C
receive event: kd D
receive event: ku D
receive event: kd F
receive event: ku F
receive event: t 10271
receive event: t 10848
receive event: t 11443
```

能看到程序输出时间事件的信息, 且敲击按键时成功输出了按键事件的信息。

### 4.3 在 NEMU 中运行仙剑奇侠传:

更新 ramdisk 之后, 在 Nanos-lite 中加载并运行/bin/pal:





运行成功！

## 5 思考题与必答题

### 5.1 思考题

#### 1. 对比异常与函数调用：

我们知道进行函数调用的时候也需要保存调用者的状态: 返回地址, 以及调用约定 (calling convention) 中需要调用者保存的寄存器, 而进行异常处理之前却要保存更多的信息. 尝试对比它们, 并思考两者保存信息不同是什么原因造成的.

回答:

函数调用和异常处理都属于控制流转移, 二者都需要保存一些信息, 以便将来能回到原来的执行位置。但二者保存的信息数量和保存责任不同, 根本原因是: 函数调用是程序主动发起的、遵守调用约定的同步控制转移; 异常或中断是由 CPU、设备或 int 指令触发的特殊控制转移, 处理程序必须能够完整恢复被打断程序的现场。

函数调用时, 调用者知道自己正在调用一个函数, 编译器和 ABI 也知道哪些寄存器允许被调用者破坏, 哪些寄存器必须由调用者或被调用者保存。因此函数调用只需要保存满足调用约定的最小状态。

以普通 near call 为例, 核心行为可以理解为:

```

1  call target:
2      push return_eip
3      eip = target
4
5  ret:
6      pop eip
7

```

在函数调用中, 返回地址由 call 指令保存, 参数和局部变量由编译器按照栈帧规则组织。对于寄存器, 调用约定会规定 caller-saved 和 callee-saved。caller-saved 寄存器被调用函数修改是允许的, 调用者如果还需要这些值, 就必须自己提前保存; callee-saved 寄存器则由被调用函数负责恢复。



也就是说，函数调用并不要求保存完整 CPU 现场。因为调用者是主动让出控制流的，它知道函数调用会发生，也接受调用约定中规定的寄存器破坏规则。

异常处理不同。异常可能发生在任意指令处，例如除零、缺页、非法指令，也可能由系统调用 `int 0x80` 主动触发。对于被打断的程序来说，异常处理程序不是普通被调用函数；程序通常希望异常处理结束后还能像没有发生过一样继续执行。因此，处理异常前必须保存更多现场。

根据 i386 手册，中断或异常进入处理程序时，CPU 会自动在栈上保存返回所需的硬件现场。无特权级切换时，主要包括：

```
1  EFLAGS
2  CS
3  EIP
4  可能还有 Error Code
5
```

在本实验的 NEMU 实现中，`raise_intr()` 模拟了这部分硬件行为：

```
1  rtlreg_t val = cpu.eflags.val;
2  rtl_push(&val);
3  val = cpu.cs;
4  rtl_push(&val);
5  val = ret_addr;
6  rtl_push(&val);
7
8  cpu.cs = (low >> 16) & 0xffff;
9  decoding.jump_eip = (low & 0xffff) | (high & 0xffff0000);
10 decoding.is_jump = 1;
11
```

这些内容和函数调用中的返回地址类似，都是为了将来返回。但异常还需要保存 EFLAGS 和 CS，因为异常返回需要恢复被打断时的标志位和代码段。EFLAGS 中包含 IF、方向标志、条件码等状态，如果不恢复，程序的后续执行就可能发生变化。CS 则描述了返回位置所在的代码段。

此外，一部分异常还会产生错误码，用于说明异常原因，例如段相关异常或页错误。PA3 中 `vecsys` 和 `vecnull` 为了统一 `trapframe` 格式，会软件压入错误码和 `irq` 号：

```
1  .globl vecsys;  vecsys:  pushl $0;  pushl $0x80; jmp asm_trap
2  .globl vecnull; vecnull: pushl $0;  pushl  $-1; jmp asm_trap
3
```

接着，`trap.S` 中的 `pushal` 会保存通用寄存器：

```
1  asm_trap:
2      pushal
3
```

i386 手册中 `PUSHAD` 的压栈顺序是保存 EAX、ECX、EDX、EBX、original ESP、EBP、ESI、EDI。由于栈向低地址增长，`pushal` 完成后，`trapframe` 从当前 `esp` 开始看到的顺序正好是 EDI、ESI、EBP、original ESP、EBX、EDX、ECX、EAX，再往后是 `irq`、error code、EIP、CS、EFLAGS。

异常处理要保存通用寄存器，是因为异常处理程序本身也要运行代码，甚至还会调用 C 函数 `irq_handle()`、

do\_event()、do\_syscall()。这些代码一定会使用 and 修改寄存器。如果不提前保存，被打断程序原来的寄存器值就会被破坏，iret 返回后程序将无法继续正确执行。

因此，二者保存信息不同的原因可以总结如下：

```

1  函数调用：
2      主动发生
3      调用点明确
4      遵守 ABI 调用约定
5      只保存返回地址和调用约定要求保存的寄存器
6      被调用函数可以合法破坏 caller-saved 寄存器
7
8  异常处理：
9      可能在任意指令处发生
10     被打断程序通常没有准备
11     处理程序不是普通业务函数
12     必须能恢复完整执行现场
13     需要保存 EIP、CS、EFLAGS、错误码、异常号、通用寄存器等信息
14

```

也就是说，函数调用保存的是“函数返回需要的最小上下文”，异常处理保存的是“被打断程序继续执行需要的完整现场”。

## 2. 诡异的代码

trap.S 中有一行 pushl %esp 的代码，乍看之下其行为十分诡异。你能结合前后的代码理解它的行为吗？Hint：不用想太多，其实都是你学过的知识。

回答：

trap.S 的关键代码如下：

```

1  #---|-----entry-----|errorcode|---irq id---|---handler---|
2  .globl vecsys;  vecsys:  pushl $0;  pushl $0x80; jmp asm_trap
3  .globl vecnull; vecnull:  pushl $0;  pushl  $-1; jmp asm_trap
4
5  asm_trap:
6  pushal
7
8  pushl %esp
9  call irq_handle
10
11  addl $4, %esp
12
13  popal
14  addl $8, %esp
15
16  iret
17

```

这行 pushl %esp 的作用其实很直接：它是在按照 x86 cdecl 调用约定给 C 函数 irq\_handle() 传



递第一个参数。这个参数就是当前 trapframe 的地址。

异常进入 vecsys 后，栈上已经有 CPU 自动保存的 EFLAGS、CS、EIP。vecsys 又压入 error code 和 irq id。进入 asm\_trap 后，pushal 保存通用寄存器。此时当前 esp 正好指向整个 trapframe 的起始位置。

栈的大致布局如下：

```

1  当前 esp -> EDI
2      ESI
3      EBP
4      original ESP
5      EBX
6      EDX
7      ECX
8      EAX
9      irq
10     error code
11     EIP
12     CS
13     EFLAGS
14

```

Nexus-AM 中 irq\_handle() 的函数原型是：

```

1  _RegSet* irq_handle(_RegSet *tf)
2

```

因此，汇编代码需要把 trapframe 的地址作为参数传给 irq\_handle()。在 cdecl 中，函数参数从右到左压栈。这里只有一个参数，所以直接把当前 esp 压栈即可：

```

1  pushl %esp
2  call irq_handle
3

```

乍看起来诡异，是因为 pushl %esp 本身会改变 esp，好像压入的地址会被自己影响。但在 i386 中，push esp 压入的是执行 push 之前的 esp 值。也就是说，这里真正传入 irq\_handle() 的值，是 pushl %esp 执行前的 esp，正好是 trapframe 的起始地址。

call irq\_handle 又会继续压入返回地址。因此进入 irq\_handle() 后，它能按 C 调用约定从栈上取得参数 tf，而 tf 指向 trap.S 已经构造好的 trapframe。

irq\_handle() 返回后：

```

1  addl $4, %esp
2

```

用于弹出刚才传给 irq\_handle() 的那个参数。随后：

```

1  popal
2  addl $8, %esp
3  iret
4

```

popal 恢复通用寄存器; addl \$8 跳过软件压入的 irq id 和 error code; iret 根据硬件保存的 EIP、CS、EFLAGS 返回到被中断的位置。

所以 pushl %esp 并不是为了保存 esp 本身, 而是为了把当前栈上 trapframe 的地址作为参数传给 C 语言的异常处理函数。

## 5.2 必答题

仙剑奇侠传中有以下行为:

- 1. 在 navy-apps/apps/pal/src/global/global.c 的 PAL\_LoadGame() 中通过 fread() 读取游戏存档。
- 2. 在 navy-apps/apps/pal/src/hal/hal.c 的 redraw() 中通过 NDL\_DrawRect() 更新屏幕。

请结合代码解释仙剑奇侠传、库函数、libos、Nanos-lite、AM、NEMU 是如何相互协助, 来分别完成游戏存档的读取和屏幕的更新。

.

回答:

### 5.2.1 游戏存档读取过程

仙剑奇侠传读取存档的入口在 PAL\_LoadGame()。相关代码如下:

```

1  static INT
2  PAL_LoadGame(
3      LPCSTR      szFileName
4  )
5  {
6      FILE          *fp;
7      PAL_LARGE SAVEDGAME    s;
8      UINT32        i;
9
10     fp = fopen(szFileName, "rb");
11     if (fp == NULL)
12     {
13         return -1;
14     }
15
16     fread(&s, sizeof(SAVEDGAME), 1, fp);
17     fclose(fp);
18
19     DO_BYTESWAP(&s, sizeof(SAVEDGAME));
20
21     gpGlobals->viewport = PAL_XY(s.wViewportX, s.wViewportY);
22     ...
23 }
24
```

调用 `PAL_LoadGame()` 时, 存档路径由 `PAL_SAVE_PREFIX` 拼出。在当前 Navy-apps 的 PAL 配置中, `PAL_PREFIX` 是 `/share/games/pal/`, `PAL_SAVE_PREFIX` 等于 `PAL_PREFIX`, 因此存档可能是 `/share/games/pal/1.rpg` 这样的路径。

整个读取流程可以分成以下几层。

第一层: PAL 应用层

PAL 只知道自己要打开一个普通文件, 并从中读出一个 `SAVEDGAME` 结构体。它调用的是 C 标准库接口 `fopen()`、`fread()`、`fclose()`。PAL 不关心文件位于 `ramdisk` 的哪个偏移, 也不关心系统调用和中断怎么实现。

伪代码如下:

```
1 PAL_LoadGame(path):
2     fp = fopen(path, "rb")
3     fread(&s, sizeof(SAVEDGAME), 1, fp)
4     fclose(fp)
5     根据 s 恢复游戏全局状态
6
```

第二层: libc 文件流层

`fopen()` 在 libc 中会申请并初始化 `FILE` 结构, 然后调用 `_open_r()` 打开底层文件描述符。`FILE` 中会设置底层读写函数指针, 例如 `_read = __sread`。

相关代码逻辑如下:

```
1 fp = __sfp(...)
2 f = _open_r(..., file, oflags, 0666)
3
4 fp->_file = f
5 fp->_read = __sread
6 fp->_write = __swrite
7 fp->_seek = __sseek
8 fp->_close = __sclose
9
```

`fread()` 本身先尝试从 `FILE` 的缓冲区中复制数据。如果缓冲区数据不够, 就调用 `__srefill()` 填充缓冲区。`__srefill()` 最终会通过 `fp->_read` 读取底层文件。

`fread()` 的核心行为可以概括为:

```
1 fread(buf, size, count, fp):
2     resid = size * count
3     while resid > fp->_r:
4         从 fp 内部缓冲区复制已有数据到 buf
5         调用 __srefill(fp) 填充缓冲区
6         从缓冲区复制剩余数据到 buf
7         返回成功读取的元素个数
8
```

`__srefill()` 中的底层读取逻辑是:

```
1 fp->_p = fp->_bf._base
```

```
fp->_r = (*fp->_read)(fp->_cookie, fp->_p, fp->_bf._size)
```

而 `__sread()` 会进一步调用 `__read_r()`，最终进入 `libos` 的 `__read()`。

第三层：libos 系统调用封装层

`libos/src/nanos.c` 把 POSIX 风格的 `_open()`、`_read()`、`_write()`、`_lseek()` 等函数翻译成 `Nanos-lite` 的系统调用。系统调用通过 `int $0x80` 进入内核。

关键代码如下：

```
int _syscall_(int type, uintptr_t a0, uintptr_t a1, uintptr_t a2){
    int ret = -1;
    asm volatile("int $0x80": "=a"(ret): "a"(type), "b"(a0), "c"(a1), "d"(a2));
    return ret;
}

int _open(const char *path, int flags, mode_t mode) {
    return _syscall_(SYS_open, (uintptr_t)path, flags, mode);
}

int _read(int fd, void *buf, size_t count) {
    return _syscall_(SYS_read, fd, (uintptr_t)buf, count);
}
```

这里的寄存器约定是：eax 放系统调用号，ebx、ecx、edx 放三个参数。执行 `int $0x80` 后，用户程序进入异常处理流程。

第四层：NEMU 和 AM 的中断/异常转发

NEMU 执行 `int` 指令时，会通过 `raise_intr()` 模拟 i386 的中断门行为：根据 IDT 找到 `0x80` 号中断处理入口，保存 EFLAGS、CS、EIP，然后跳转到 `vecsys`。

Nexus-AM 的 `_asye_init()` 初始化 IDT，并把 `0x80` 号中断设置为系统调用入口 `vecsys`：

```
idt[0x80] = GATE(STS_TG32, KSEL(SEG_KCODE), vecsys, DPL_USER);
set_idt(idt, sizeof(idt));
```

`trap.S` 中 `vecsys` 和 `asm_trap` 构造 `trapframe`，然后调用 `irq_handle()`：

```
vecsys:
    pushl $0
    pushl $0x80
    jmp asm_trap

asm_trap:
    pushal
    pushl %esp
    call irq_handle
    ...
```

irq\_handle() 根据 trapframe 中的 irq 号判断这是系统调用事件:

```

1  switch (tf->irq) {
2      case 0x80: ev.event = _EVENT_SYSCALL; break;
3      default: ev.event = _EVENT_ERROR; break;
4  }
5

```

随后 Nanos-lite 在 do\_event() 中识别 \_EVENT\_SYSCALL, 并调用 do\_syscall()。

第五层: Nanos-lite 系统调用和文件系统层

Nanos-lite 的 do\_syscall() 从 trapframe 中取出系统调用号和参数, 然后分发到文件系统函数。

```

1  switch (a[0]) {
2      case SYS_open:
3          SYSCALL_ARG1(r) = fs_open((const char *)a[1], a[2], a[3]);
4          break;
5      case SYS_read:
6          SYSCALL_ARG1(r) = fs_read(a[1], (void *)a[2], a[3]);
7          break;
8      case SYS_close:
9          SYSCALL_ARG1(r) = fs_close(a[1]);
10         break;
11     }
12

```

当 PAL 调用 fopen() 时, 最终触发 SYS\_open。fs\_open() 在 file\_table 中查找路径对应的文件记录, 并返回文件描述符。

当 PAL 调用 fread() 时, 最终触发 SYS\_read。fs\_read() 根据文件描述符取得 Finfo, 计算当前 open\_offset 和文件大小, 调用 ramdisk\_read() 从 ramdisk 中复制数据。

核心逻辑如下:

```

1  ssize_t fs_read(int fd, void *buf, size_t len) {
2      Finfo *file = &file_table[fd];
3      size_t open_offset = (size_t)file->open_offset;
4
5      if (len > file->size - open_offset) {
6          len = file->size - open_offset;
7      }
8
9      ramdisk_read(buf, file->disk_offset + file->open_offset, len);
10     file->open_offset += len;
11     return len;
12 }
13

```

ramdisk\_read() 则直接从内核镜像中的 ramdisk 区域复制字节:

```

1  void ramdisk_read(void *buf, off_t offset, size_t len) {
2      assert(offset + len <= RAMDISK_SIZE);

```

```

3     memcpy(buf, &ramdisk_start + offset, len);
4 }
5

```

这里没有复杂的磁盘驱动。PA3 的 ramdisk 是 make update 时由 Navy-apps/fsimg 打包并链接进 Nanos-lite 的。src/files.h 中记录了每个文件的文件名、大小和在 ramdisk 中的偏移，fs\_open() 和 fs\_read() 就依靠这些信息完成文件访问。

第六层：返回用户程序

fs\_read() 返回读到的字节数，do\_syscall() 把返回值写回 trapframe 中的 eax。之后 trap.S 用 popal 恢复通用寄存器，用 iret 恢复 EIP、CS、EFLAGS，用户态的 int \$0x80 返回。libos 的 \_read() 得到返回值，libc 的 fread() 继续把缓冲区数据复制到 PAL 的 SAVEDGAME 变量 s 中。最后 PAL 根据 s 恢复游戏进度、场景、队伍、物品等全局状态。

完整链路可以概括为：

```

1 PAL_LoadGame()
2 -> fopen()
3   -> libc _open_r()
4     -> libos _open()
5       -> int $0x80
6         -> NEMU raise_intr()
7           -> AM vecsys / asm_trap / irq_handle()
8             -> Nanos-lite do_event()
9               -> do_syscall(SYS_open)
10                 -> fs_open()
11
12 PAL_LoadGame()
13 -> fread()
14   -> libc __srefill()
15     -> __sread()
16       -> libos _read()
17         -> int $0x80
18           -> NEMU raise_intr()
19             -> AM trap handling
20               -> Nanos-lite do_syscall(SYS_read)
21                 -> fs_read()
22                   -> ramdisk_read()
23                     -> memcpy() from ramdisk to user buffer
24

```

在这个过程中，PAL 只使用标准 C 文件接口；libc 负责 FILE 缓冲；libos 负责把 POSIX 风格接口转成系统调用；Nanos-lite 负责系统调用分发和 ramdisk 文件系统；AM 负责中断入口和 trapframe；NEMU 负责模拟 CPU 执行、int/iret 和内存访问。

### 5.2.2 屏幕更新过程

仙剑奇侠传更新屏幕的入口在 hal.c 的 redraw()。相关代码如下：

```

1 static void redraw() {

```

```

2   for (int i = 0; i < W; i++)
3   for (int j = 0; j < H; j++)
4     fb[i + j * W] = palette[vmem[i + j * W]];
5
6   NDL_DrawRect(fb, 0, 0, W, H);
7   NDL_Render();
8 }
9

```

PAL 的内部显存 vmem 是 8 位调色板索引，redraw() 先用 palette 把每个像素转换成 32 位颜色，写入 fb 数组。然后调用 NDL\_DrawRect() 把整屏像素提交给 NDL，再调用 NDL\_Render() 真正刷新到 /dev/fb。

第一层：PAL 应用层

PAL 并不直接操作 VGA 显存。它只维护自己的 vmem 和 palette，并调用 NDL 接口。

```

1 redraw():
2   for each pixel:
3     fb[pixel] = palette[vmem[pixel]]
4   NDL_DrawRect(fb, 0, 0, W, H)
5   NDL_Render()
6

```

第二层：NDL 图形库层

NDL\_OpenDisplay() 会先读取 /proc/dispinfo 获取屏幕大小，然后打开 /dev/fb 和 /dev/events。相关逻辑如下：

```

1 get_display_info();
2 pad_x = (screen_w - canvas_w) / 2;
3 pad_y = (screen_h - canvas_h) / 2;
4 fbdev = fopen("/dev/fb", "w");
5 evtdev = fopen("/dev/events", "r");
6

```

get\_display\_info() 通过 /proc/dispinfo 读取屏幕大小：

```

1 FILE *dispinfo = fopen("/proc/dispinfo", "r");
2 while (fgets(buf, 128, dispinfo)) {
3     *(delim = strchr(buf, ':')) = '\0';
4     sscanf(buf, "%s", key);
5     sscanf(delim + 1, "%s", value);
6     if (strcmp(key, "WIDTH") == 0) sscanf(value, "%d", &screen_w);
7     if (strcmp(key, "HEIGHT") == 0) sscanf(value, "%d", &screen_h);
8 }
9

```

这里的 /proc/dispinfo 由 Nanos-lite 的 device.c 生成：

```

1 void init_device() {
2     _ioe_init();
3

```

```

3     snprintf(dispinfo, sizeof(dispinfo), "WIDTH: %d\nHEIGHT: %d\n",
4             _screen.width, _screen.height);
5 }
6

```

NDL\_DrawRect() 本身只是把传入的像素复制到 NDL 的 canvas 中：

```

1 int NDL_DrawRect(uint32_t *pixels, int x, int y, int w, int h) {
2     for (int i = 0; i < h; i++) {
3         for (int j = 0; j < w; j++) {
4             canvas[(i + y) * canvas_w + (j + x)] = pixels[i * w + j];
5         }
6     }
7 }
8

```

真正写入 /dev/fb 的动作发生在 NDL\_Render() 中。它逐行设置 /dev/fb 的文件偏移，然后通过 fwrite() 写入一行像素：

```

1 int NDL_Render() {
2     for (int i = 0; i < canvas_h; i++) {
3         fseek(fbdev, ((i + pad_y) * screen_w + pad_x) * sizeof(uint32_t), SEEK_SET);
4         fwrite(&canvas[i * canvas_w], sizeof(uint32_t), canvas_w, fbdev);
5     }
6     fflush(fbdev);
7 }
8

```

因此，题目中说通过 NDL\_DrawRect() 更新屏幕，完整过程还包括紧随其后的 NDL\_Render()。NDL\_DrawRect() 准备 canvas，NDL\_Render() 才把 canvas 写入 /dev/fb。

第三层：libc 和 libos 层

NDL\_Render() 中的 fseek() 和 fwrite() 属于 libc 文件接口。它们最终分别调用 libos 的 \_lseek() 和 \_write()。

libos 中的封装如下：

```

1 int _write(int fd, void *buf, size_t count){
2     return _syscall_(SYS_write, fd, (uintptr_t)buf, count);
3 }
4
5 off_t _lseek(int fd, off_t offset, int whence) {
6     return _syscall_(SYS_lseek, fd, offset, whence);
7 }
8

```

这些系统调用同样通过 int \$0x80 进入 Nanos-lite。中断入口、trapframe 构造、irq\_handle() 和 do\_event() 的过程与存档读取相同。

第四层：Nanos-lite 文件系统和设备文件层

Nanos-lite 把 VGA 显存抽象为 /dev/fb。文件表中为它保留了特殊 fd：



```

1 enum {FD_STDIN, FD_STDOUT, FD_STDERR, FD_FB, FD_EVENTS, FD_DISPINFO, FD_NORMAL};
2
3 [FD_FB] = {"/dev/fb", 0, 0},
4 [FD_EVENTS] = {"/dev/events", 0, 0},
5 [FD_DISPINFO] = {"/proc/dispinfo", 128, 0},
6

```

init\_fs() 根据 AM 提供的屏幕大小初始化 /dev/fb 的大小:

```

1 void init_fs() {
2     file_table[FD_FB].size = _screen.width * _screen.height * sizeof(uint32_t);
3 }
4

```

当 NDL\_Render() 调用 fseek() 时, 最终进入 fs\_lseek()。fs\_lseek() 修改 /dev/fb 的 open\_offset, 用于表示下一次写入的帧缓冲偏移。

关键逻辑如下:

```

1 case SEEK_SET:
2     new_offset = offset;
3     break;
4 case SEEK_CUR:
5     new_offset = file->open_offset + offset;
6     break;
7 case SEEK_END:
8     new_offset = (off_t)file->size + offset;
9     break;
10
11 file->open_offset = new_offset;
12

```

当 NDL\_Render() 调用 fwrite() 时, 最终进入 fs\_write()。fs\_write() 发现 fd 是 FD\_FB, 就不写 ramdisk, 而是调用 fb\_write()。

关键逻辑如下:

```

1 if (fd == FD_FB) {
2     Finfo *file = &file_table[fd];
3     size_t open_offset = (size_t)file->open_offset;
4     if (len > file->size - open_offset) {
5         len = file->size - open_offset;
6     }
7     fb_write(buf, file->open_offset, len);
8     file->open_offset += len;
9     return len;
10 }
11

```

fb\_write() 把 /dev/fb 的字节偏移转换为屏幕坐标, 然后调用 AM 的 \_draw\_rect()。因为一次写

入可能跨越行边界，所以实现中按行绘制。

关键逻辑如下：

```

1 void fb_write(const void *buf, off_t offset, size_t len) {
2     const uint32_t *pixels = buf;
3     size_t pixel_offset = (size_t)offset / sizeof(uint32_t);
4     size_t nr_pixels = len / sizeof(uint32_t);
5
6     while (nr_pixels > 0) {
7         int x = pixel_offset % _screen.width;
8         int y = pixel_offset / _screen.width;
9
10        size_t row_pixels = _screen.width - x;
11        if (row_pixels > nr_pixels) {
12            row_pixels = nr_pixels;
13        }
14        _draw_rect(pixels, x, y, row_pixels, 1);
15
16        pixels += row_pixels;
17        pixel_offset += row_pixels;
18        nr_pixels -= row_pixels;
19    }
20    _draw_sync();
21 }
22

```

第五层：AM 抽象机器层

AM 的 x86-nemu IOE 实现中，`_screen` 保存屏幕大小，`_draw_rect()` 把像素复制到地址 0x40000 开始的帧缓冲区域。

```

1 uint32_t* const fb = (uint32_t *)0x40000;
2
3 _Screen _screen = {
4     .width = 400,
5     .height = 300,
6 };
7
8 void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h) {
9     int cp_bytes = sizeof(uint32_t) * min(w, _screen.width - x);
10    for (int j = 0; j < h && y + j < _screen.height; j++) {
11        memcpy(&fb[(y + j) * _screen.width + x], pixels, cp_bytes);
12        pixels += w;
13    }
14 }
15

```

AM 层的作用是把 Nanos-lite 的设备请求转换为对抽象硬件的访问。Nanos-lite 不需要知道 NEMU 内部如何用 SDL 显示窗口，它只需要调用 `_draw_rect()`。

第六层：NEMU 设备模拟层

NEMU 中, VGA 显存地址 0x40000 被注册为 MMIO 区域:

```

1  #define VMEM 0x40000
2
3  void init_vga() {
4      SDL_CreateWindowAndRenderer(SCREEN_W * 2, SCREEN_H * 2, 0, &window, &renderer);
5      texture = SDL_CreateTexture(renderer, SDL_PIXELFORMAT_ARGB8888,
6      SDL_TEXTUREACCESS_STATIC, SCREEN_W, SCREEN_H);
7
8      vmem = add_mmio_map(VMEM, 0x80000, vga_vmem_io_handler);
9  }
10

```

当 AM 的 `_draw_rect()` 对 0x40000 附近地址执行 `memcpy()` 时, 本质上是客户程序在 NEMU 模拟的 CPU 中执行内存写指令。NEMU 的 `paddr_write()` 会检查目标地址是否属于 MMIO:

```

1  void paddr_write(paddr_t addr, int len, uint32_t data) {
2      int map_NO = is_mmio(addr);
3      if (map_NO != -1) {
4          mmio_write(addr, len, data, map_NO);
5          return;
6      }
7      memcpy(guest_to_host(addr), &data, len);
8  }
9

```

如果是 MMIO 地址, 就写入 vga 设备对应的 `mmio_space`。NEMU 的 `device_update()` 会周期性调用 `update_screen()`, 把 `vmem` 中的像素通过 SDL 显示到宿主窗口中。

关键代码如下:

```

1  void update_screen() {
2      SDL_UpdateTexture(texture, NULL, vmem, SCREEN_W * sizeof(vmem[0][0]));
3      SDL_RenderClear(renderer);
4      SDL_RenderCopy(renderer, texture, NULL, NULL);
5      SDL_RenderPresent(renderer);
6  }
7

```

完整的屏幕更新链路可以概括为:

```

1  PAL redraw()
2  -> 将 8 位 vmem 调色板索引转换为 32 位 fb 像素
3  -> NDL_DrawRect(fb, 0, 0, W, H)
4  -> 复制到 NDL canvas
5  -> NDL_Render()
6  -> fseek(/dev/fb, row_offset)
7  -> libos _lseek()
8  -> int $0x80
9  -> Nanos-lite do_syscall(SYS_lseek)
10 -> fs_lseek() 更新 FD_FB.open_offset

```

```
11 -> fwrite(row_pixels, /dev/fb)
12 -> libos _write()
13 -> int $0x80
14 -> Nanos-lite do_syscall(SYS_write)
15 -> fs_write()
16 -> fb_write()
17 -> AM _draw_rect()
18 -> 写入 0x40000 VGA MMIO
19 -> NEMU mmio_write()
20 -> NEMU update_screen()
21 -> SDL 显示窗口刷新
22
```

在这个过程中，PAL 只看到 NDL 图形接口；NDL 把图形操作转换成对 `/dev/fb` 的文件写入；libc 提供 `fseek()`、`fwrite()` 缓冲和文件流接口；libos 把文件操作翻译成系统调用；Nanos-lite 把 `/dev/fb` 识别为设备文件并调用 `fb_write()`；AM 把绘图请求转换为抽象 VGA 显存写入；NEMU 负责模拟 MMIO 显存，并最终用 SDL 在宿主机窗口显示图像。